

LAB TASK # 10

Mian Daniyal Hassan

SAP ID # 70150006

; Complete Task 10 Solution - Decimal and Hex Conversion Program

org 100h

.data

menu_msg db 13, 10, 'Number Conversion Program', 13, 10

db '1. Hexadecimal to Decimal', 13, 10

db '2. Decimal to Hexadecimal', 13, 10

db 'Enter your choice (1-2): \$'

hex_prompt db 13, 10, 'Enter hex number (max 4 digits): \$'

dec_prompt db 13, 10, 'Enter decimal number (max 65535): \$'

result_msg db 13, 10, 'Result: \$'

error_msg db 13, 10, 'Invalid input! \$'

newline db 13, 10, '\$'

hex_input db 5 dup(0) ; 4 digits + null terminator

dec_input db 6 dup(0) ; 5 digits + null terminator

result_buffer db 6 dup(0) ; Buffer for result

temp_value dw 0 ; Temporary storage for calculations

.code

main proc

mov ax, @data

mov ds, ax

menu:

; Display menu

mov dx, offset menu_msg

mov ah, 9

int 21h

; Get choice

mov ah, 1

int 21h

cmp al, '1'

je hex_to_dec

cmp al, '2'

je dec_to_hex

jmp exit

hex_to_dec:

call hex_decimal_conversion

jmp menu

dec_to_hex:

call decimal_hex_conversion

jmp menu

exit:

mov ah, 4ch

int 21h

main endp

; Procedure for Hex to Decimal conversion

hex_decimal_conversion proc

; Display prompt

mov dx, offset hex_prompt

```
mov ah, 9
```

```
int 21h
```

```
; Read hex input
```

```
mov si, offset hex_input
```

```
mov cx, 0      ; character counter
```

```
read_hex:
```

```
mov ah, 1
```

```
int 21h
```

```
cmp al, 13     ; Check for Enter key
```

```
je end_hex_read
```

```
; Validate hex digit
```

```
cmp al, '0'
```

```
jb invalid_hex
```

```
cmp al, '9'
```

```
jbe valid_hex_digit
```

```
cmp al, 'A'
```

```
jb invalid_hex
```

```
cmp al, 'F'
```

```
ja invalid_hex
```

```
valid_hex_digit:
```

```
mov [si], al
```

```
inc si
```

```
inc cx
```

```
cmp cx, 4      ; Max 4 digits
```

jbe read_hex

end_hex_read:

mov byte [si], 0 ; Null terminate

; Convert hex string to number

mov si, offset hex_input

mov ax, 0 ; Accumulator

mov bx, 0 ; Current digit

convert_hex:

mov bl, [si]

cmp bl, 0

je display_decimal

; Convert character to value

cmp bl, '9'

jbe hex_digit

sub bl, 'A'

add bl, 10

jmp process_hex

hex_digit:

sub bl, '0'

process_hex:

shl ax, 4 ; Multiply by 16

add al, bl

inc si

jmp convert_hex

display_decimal:

mov temp_value, ax

; Display result message

mov dx, offset result_msg

mov ah, 9

int 21h

; Convert to decimal string and display

mov ax, temp_value

call display_num

mov dx, offset newline

mov ah, 9

int 21h

ret

invalid_hex:

mov dx, offset error_msg

mov ah, 9

int 21h

ret

hex_decimal_conversion endp

; Procedure for Decimal to Hex conversion

decimal_hex_conversion proc

; Display prompt

mov dx, offset dec_prompt

mov ah, 9

int 21h

; Read decimal input

mov si, offset dec_input

mov cx, 0 ; digit counter

read_decimal:

mov ah, 1

int 21h

cmp al, 13 ; Check for Enter key

je end_dec_read

; Validate decimal digit

cmp al, '0'

jb invalid_decimal

cmp al, '9'

ja invalid_decimal

mov [si], al

inc si

inc cx

cmp cx, 5 ; Max 5 digits

jbe read_decimal

end_dec_read:

mov byte [si], 0 ; Null terminate

; Convert decimal string to number

```
mov si, offset dec_input
mov ax, 0      ; Accumulator
mov bx, 10     ; Multiplier
```

convert_decimal:

```
mov cl, [si]
cmp cl, 0
je convert_to_hex
```

```
sub cl, '0'
mul bx      ; AX = AX * 10
add al, cl
adc ah, 0
inc si
jmp convert_decimal
```

convert_to_hex:

```
mov temp_value, ax
```

```
; Display result message
```

```
mov dx, offset result_msg
```

```
mov ah, 9
```

```
int 21h
```

```
; Convert to hex string and display
```

```
mov ax, temp_value
```

```
call display_hex
```

```
mov dx, offset newline
```

```
mov ah, 9
```

```
int 21h
```

```
ret
```

```
invalid_decimal:
```

```
mov dx, offset error_msg
```

```
mov ah, 9
```

```
int 21h
```

```
ret
```

```
decimal_hex_conversion endp
```

```
; Procedure to display number in decimal (AX)
```

```
display_num proc
```

```
push ax
```

```
push bx
```

```
push cx
```

```
push dx
```

```
mov bx, 10      ; divisor
```

```
mov cx, 0       ; counter for digits
```

```
; Handle zero case
```

```
cmp ax, 0
```

```
jne divide_loop
```

```
push 0
```

```
inc cx
```

```
jmp display_digits
```

```
divide_loop:
```

```
mov dx, 0      ; clear high word of dividend
div bx         ; divide by 10
push dx        ; push remainder
inc cx         ; increment digit counter
cmp ax, 0      ; check if quotient is zero
jne divide_loop
```

display_digits:

```
pop dx         ; pop digit
add dl, '0'    ; convert to ASCII
mov ah, 2      ; display character function
int 21h
loop display_digits
```

```
pop dx
pop cx
pop bx
pop ax
ret
```

display_num endp

; Procedure to display number in hex (AX)

display_hex proc

```
push ax
push bx
push cx
push dx
```

```
mov cx, 4      ; four hex digits
```

```
mov bx, ax    ; save number
```

```
display_hex_loop:
```

```
rol bx, 4     ; rotate left by 4 bits
```

```
mov al, bl    ; move lowest byte to al
```

```
and al, 0Fh   ; mask off high nibble
```

```
cmp al, 9     ; convert to ASCII
```

```
jbe digit_to_ascii
```

```
add al, 'A'-10
```

```
jmp show_hex
```

```
digit_to_ascii:
```

```
add al, '0'
```

```
show_hex:
```

```
mov dl, al
```

```
mov ah, 2
```

```
int 21h
```

```
loop display_hex_loop
```

```
pop dx
```

```
pop cx
```

```
pop bx
```

```
pop ax
```

```
ret
```

```
display_hex endp
```

```
end main
```

